

Prototypical Networks for Few-shot Learning

Jake Snell
University of Toronto*

Kevin Swersky
Twitter

Richard S. Zemel
University of Toronto, Vector Institute

Abstract

We propose *prototypical networks* for the problem of few-shot classification, where a classifier must generalize to new classes not seen in the training set, given only a small number of examples of each new class. Prototypical networks learn a metric space in which classification can be performed by computing distances to prototype representations of each class. Compared to recent approaches for few-shot learning, they reflect a simpler inductive bias that is beneficial in this limited-data regime, and achieve excellent results. We provide an analysis showing that some simple design decisions can yield substantial improvements over recent approaches involving complicated architectural choices and meta-learning. We further extend prototypical networks to zero-shot learning and achieve state-of-the-art results on the CU-Birds dataset.

1 Introduction

Few-shot classification [20, 16, 13] is a task in which a classifier must be adapted to accommodate new classes not seen in training, given only a few examples of each of these classes. A naive approach, such as re-training the model on the new data, would severely overfit. While the problem is quite difficult, it has been demonstrated that humans have the ability to perform even one-shot classification, where only a single example of each new class is given, with a high degree of accuracy [16].

Two recent approaches have made significant progress in few-shot learning. Vinyals et al. [29] proposed *matching networks*, which uses an attention mechanism over a learned embedding of the labeled set of examples (the *support set*) to predict classes for the unlabeled points (the *query set*). Matching networks can be interpreted as a weighted nearest-neighbor classifier applied within an embedding space. Notably, this model utilizes sampled mini-batches called *episodes* during training, where each episode is designed to mimic the few-shot task by subsampling classes as well as data points. The use of episodes makes the training problem more faithful to the test environment and thereby improves generalization. Ravi and Larochelle [22] take the episodic training idea further and propose a meta-learning approach to few-shot learning. Their approach involves training an LSTM [9] to produce the updates to a classifier, given an episode, such that it will generalize well to a test-set. Here, rather than training a single model over multiple episodes, the LSTM meta-learner learns to train a custom model for each episode.

We attack the problem of few-shot learning by addressing the key issue of overfitting. Since data is severely limited, we work under the assumption that a classifier should have a very simple inductive bias. Our approach, *prototypical networks*, is based on the idea that there exists an embedding in which points cluster around a single prototype representation for each class. In order to do this, we learn a non-linear mapping of the input into an embedding space using a neural network and take a class's prototype to be the mean of its support set in the embedding space. Classification is then performed for an embedded query point by simply finding the nearest class prototype. We follow the same approach to tackle zero-shot learning; here each class comes with meta-data giving a high-level description of the class rather than a small number of labeled examples. We therefore learn an embedding of the meta-data into a shared space to serve as the prototype for each class.

*Initial work by first author done while at Twitter.

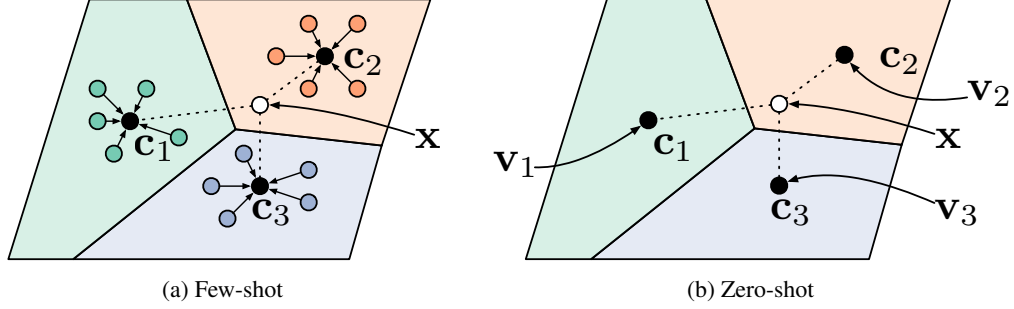


Figure 1: Prototypical networks in the few-shot and zero-shot scenarios. **Left:** Few-shot prototypes $\mathbf{c}_k$ are computed as the mean of embedded support examples for each class. **Right:** Zero-shot prototypes $\mathbf{c}_k$ are produced by embedding class meta-data $\mathbf{v}_k$. In either case, embedded query points are classified via a softmax over distances to class prototypes: $p_\phi(y = k|\mathbf{x}) \propto \exp(-d(f_\phi(\mathbf{x}), \mathbf{c}_k))$.

Classification is performed, as in the few-shot scenario, by finding the nearest class prototype for an embedded query point.

In this paper, we formulate prototypical networks for both the few-shot and zero-shot settings. We draw connections to matching networks in the one-shot setting, and analyze the underlying distance function used in the model. In particular, we relate prototypical networks to clustering [4] in order to justify the use of class means as prototypes when distances are computed with a Bregman divergence, such as squared Euclidean distance. We find empirically that the choice of distance is vital, as Euclidean distance greatly outperforms the more commonly used cosine similarity. On several benchmark tasks, we achieve state-of-the-art performance. Prototypical networks are simpler and more efficient than recent meta-learning algorithms, making them an appealing approach to few-shot and zero-shot learning.

2 Prototypical Networks

2.1 Notation

In few-shot classification we are given a small support set of N labeled examples $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ where each $\mathbf{x}_i \in \mathbb{R}^D$ is the D -dimensional feature vector of an example and $y_i \in \{1, \dots, K\}$ is the corresponding label. S_k denotes the set of examples labeled with class k .

2.2 Model

Prototypical networks compute an M -dimensional representation $\mathbf{c}_k \in \mathbb{R}^M$, or *prototype*, of each class through an embedding function $f_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M$ with learnable parameters ϕ . Each prototype is the mean vector of the embedded support points belonging to its class:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{(\mathbf{x}_i, y_i) \in S_k} f_\phi(\mathbf{x}_i) \quad (1)$$

Given a distance function $d : \mathbb{R}^M \times \mathbb{R}^M \rightarrow [0, +\infty)$, prototypical networks produce a distribution over classes for a query point $\mathbf{x}$ based on a softmax over distances to the prototypes in the embedding space:

$$p_\phi(y = k | \mathbf{x}) = \frac{\exp(-d(f_\phi(\mathbf{x}), \mathbf{c}_k))}{\sum_{k'} \exp(-d(f_\phi(\mathbf{x}), \mathbf{c}_{k'}))} \quad (2)$$

Learning proceeds by minimizing the negative log-probability $J(\phi) = -\log p_\phi(y = k | \mathbf{x})$ of the true class k via SGD. Training episodes are formed by randomly selecting a subset of classes from the training set, then choosing a subset of examples within each class to act as the support set and a subset of the remainder to serve as query points. Pseudocode to compute the loss $J(\phi)$ for a training episode is provided in Algorithm 1.

Algorithm 1 Training episode loss computation for prototypical networks. N is the number of examples in the training set, K is the number of classes in the training set, $N_C \leq K$ is the number of classes per episode, N_S is the number of support examples per class, N_Q is the number of query examples per class. $\text{RANDOMSAMPLE}(S, N)$ denotes a set of N elements chosen uniformly at random from set S , without replacement.

Input: Training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, where each $y_i \in \{1, \dots, K\}$. $\mathcal{D}_k$ denotes the subset of $\mathcal{D}$ containing all elements $(\mathbf{x}_i, y_i)$ such that $y_i = k$.

Output: The loss J for a randomly generated training episode.

```

 $V \leftarrow \text{RANDOMSAMPLE}(\{1, \dots, K\}, N_C)$  ▷ Select class indices for episode
for  $k$  in  $\{1, \dots, N_C\}$  do
   $S_k \leftarrow \text{RANDOMSAMPLE}(\mathcal{D}_{V_k}, N_S)$  ▷ Select support examples
   $Q_k \leftarrow \text{RANDOMSAMPLE}(\mathcal{D}_{V_k} \setminus S_k, N_Q)$  ▷ Select query examples
   $\mathbf{c}_k \leftarrow \frac{1}{N_C} \sum_{(\mathbf{x}_i, y_i) \in S_k} f_\phi(\mathbf{x}_i)$  ▷ Compute prototype from support examples
end for
 $J \leftarrow 0$  ▷ Initialize loss
for  $k$  in  $\{1, \dots, N_C\}$  do
  for  $(\mathbf{x}, y)$  in  $Q_k$  do
     $J \leftarrow J + \frac{1}{N_C N_Q} \left[ d(f_\phi(\mathbf{x}), \mathbf{c}_k) + \log \sum_{k'} \exp(-d(f_\phi(\mathbf{x}), \mathbf{c}_{k'})) \right]$  ▷ Update loss
  end for
end for

```

2.3 Prototypical Networks as Mixture Density Estimation

For a particular class of distance functions, known as *regular Bregman divergences* [4], the prototypical networks algorithm is equivalent to performing mixture density estimation on the support set with an exponential family density. A regular Bregman divergence d_φ is defined as:

$$d_\varphi(\mathbf{z}, \mathbf{z}') = \varphi(\mathbf{z}) - \varphi(\mathbf{z}') - (\mathbf{z} - \mathbf{z}')^T \nabla \varphi(\mathbf{z}'), \quad (3)$$

where φ is a differentiable, strictly convex function of the Legendre type. Examples of Bregman divergences include squared Euclidean distance $\|\mathbf{z} - \mathbf{z}'\|^2$ and Mahalanobis distance.

Prototype computation can be viewed in terms of hard clustering on the support set, with one cluster per class and each support point assigned to its corresponding class cluster. It has been shown [4] for Bregman divergences that the cluster representative achieving minimal distance to its assigned points is the cluster mean. Thus the prototype computation in Equation (1) yields optimal cluster representatives given the support set labels when a Bregman divergence is used.

Moreover, any regular exponential family distribution $p_\psi(\mathbf{z}|\boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ and cumulant function ψ can be written in terms of a uniquely determined regular Bregman divergence [4]:

$$p_\psi(\mathbf{z}|\boldsymbol{\theta}) = \exp\{\mathbf{z}^T \boldsymbol{\theta} - \psi(\boldsymbol{\theta}) - g_\psi(\mathbf{z})\} = \exp\{-d_\varphi(\mathbf{z}, \boldsymbol{\mu}(\boldsymbol{\theta})) - g_\varphi(\mathbf{z})\} \quad (4)$$

Consider now a regular exponential family mixture model with parameters $\boldsymbol{\Gamma} = \{\boldsymbol{\theta}_k, \pi_k\}_{k=1}^K$:

$$p(\mathbf{z}|\boldsymbol{\Gamma}) = \sum_{k=1}^K \pi_k p_\psi(\mathbf{z}|\boldsymbol{\theta}_k) = \sum_{k=1}^K \pi_k \exp(-d_\varphi(\mathbf{z}, \boldsymbol{\mu}(\boldsymbol{\theta}_k)) - g_\varphi(\mathbf{z})) \quad (5)$$

Given $\boldsymbol{\Gamma}$, inference of the cluster assignment y for an unlabeled point $\mathbf{z}$ becomes:

$$p(y = k|\mathbf{z}) = \frac{\pi_k \exp(-d_\varphi(\mathbf{z}, \boldsymbol{\mu}(\boldsymbol{\theta}_k)))}{\sum_{k'} \pi_{k'} \exp(-d_\varphi(\mathbf{z}, \boldsymbol{\mu}(\boldsymbol{\theta}_{k'})))} \quad (6)$$

For an equally-weighted mixture model with one cluster per class, cluster assignment inference (6) is equivalent to query class prediction (2) with $f_\phi(\mathbf{x}) = \mathbf{z}$ and $\mathbf{c}_k = \boldsymbol{\mu}(\boldsymbol{\theta}_k)$. In this case, prototypical networks are effectively performing mixture density estimation with an exponential family distribution determined by d_φ . The choice of distance therefore specifies modeling assumptions about the class-conditional data distribution in the embedding space.

2.4 Reinterpretation as a Linear Model

A simple analysis is useful in gaining insight into the nature of the learned classifier. When we use Euclidean distance $d(\mathbf{z}, \mathbf{z}') = \|\mathbf{z} - \mathbf{z}'\|^2$, then the model in Equation (2) is equivalent to a linear model with a particular parameterization [19]. To see this, expand the term in the exponent:

$$-\|f_\phi(\mathbf{x}) - \mathbf{c}_k\|^2 = -f_\phi(\mathbf{x})^\top f_\phi(\mathbf{x}) + 2\mathbf{c}_k^\top f_\phi(\mathbf{x}) - \mathbf{c}_k^\top \mathbf{c}_k \quad (7)$$

The first term in Equation (7) is constant with respect to the class k , so it does not affect the softmax probabilities. We can write the remaining terms as a linear model as follows:

$$2\mathbf{c}_k^\top f_\phi(\mathbf{x}) - \mathbf{c}_k^\top \mathbf{c}_k = \mathbf{w}_k^\top f_\phi(\mathbf{x}) + b_k, \text{ where } \mathbf{w}_k = 2\mathbf{c}_k \text{ and } b_k = -\mathbf{c}_k^\top \mathbf{c}_k \quad (8)$$

We focus primarily on squared Euclidean distance (corresponding to spherical Gaussian densities) in this work. Our results indicate that Euclidean distance is an effective choice despite the equivalence to a linear model. We hypothesize this is because all of the required non-linearity can be learned within the embedding function. Indeed, this is the approach that modern neural network classification systems currently use, e.g., [14, 28].

2.5 Comparison to Matching Networks

Prototypical networks differ from matching networks in the few-shot case with equivalence in the one-shot scenario. Matching networks [29] produce a weighted nearest neighbor classifier given the support set, while prototypical networks produce a linear classifier when squared Euclidean distance is used. In the case of one-shot learning, $\mathbf{c}_k = \mathbf{x}_k$ since there is only one support point per class, and matching networks and prototypical networks become equivalent.

A natural question is whether it makes sense to use multiple prototypes per class instead of just one. If the number of prototypes per class is fixed and greater than 1, then this would require a partitioning scheme to further cluster the support points within a class. This has been proposed in Mensink et al. [19] and Rippel et al. [25]; however both methods require a separate partitioning phase that is decoupled from the weight updates, while our approach is simple to learn with ordinary gradient descent methods.

Vinyals et al. [29] propose a number of extensions, including decoupling the embedding functions of the support and query points, and using a second-level, fully-conditional embedding (FCE) that takes into account specific points in each episode. These could likewise be incorporated into prototypical networks, however they increase the number of learnable parameters, and FCE imposes an arbitrary ordering on the support set using a bi-directional LSTM. Instead, we show that it is possible to achieve the same level of performance using simple design choices, which we outline next.

2.6 Design Choices

Distance metric Vinyals et al. [29] and Ravi and Larochelle [22] apply matching networks using cosine distance. However for both prototypical and matching networks any distance is permissible, and we found that using squared Euclidean distance can greatly improve results for both. We conjecture this is primarily due to cosine distance not being a Bregman divergence, and thus the equivalence to mixture density estimation discussed in Section 2.3 does not hold.

Episode composition A straightforward way to construct episodes, used in Vinyals et al. [29] and Ravi and Larochelle [22], is to choose N_c classes and N_S support points per class in order to match the expected situation at test-time. That is, if we expect at test-time to perform 5-way classification and 1-shot learning, then training episodes could be comprised of $N_c = 5$, $N_S = 1$. We have found, however, that it can be extremely beneficial to train with a higher N_c , or “way”, than will be used at test-time. In our experiments, we tune the training N_c on a held-out validation set. Another consideration is whether to match N_S , or “shot”, at train and test-time. For prototypical networks, we found that it is usually best to train and test with the same “shot” number.

2.7 Zero-Shot Learning

Zero-shot learning differs from few-shot learning in that instead of being given a support set of training points, we are given a class meta-data vector $\mathbf{v}_k$ for each class. These could be determined

Table 1: Few-shot classification accuracies on Omniglot.

Model	Dist.	Fine Tune	5-way Acc.		20-way Acc.	
			1-shot	5-shot	1-shot	5-shot
MATCHING NETWORKS [29]	Cosine	N	98.1%	98.9%	93.8%	98.5%
MATCHING NETWORKS [29]	Cosine	Y	97.9%	98.7%	93.5%	98.7%
NEURAL STATISTICIAN [6]	-	N	98.1%	99.5%	93.2%	98.1%
PROTOTYPICAL NETWORKS (OURS)	Euclid.	N	98.8%	99.7%	96.0%	98.9%

in advance, or they could be learned from e.g., raw text [7]. Modifying prototypical networks to deal with the zero-shot case is straightforward: we simply define $\mathbf{c}_k = g_{\theta}(\mathbf{v}_k)$ to be a separate embedding of the meta-data vector. An illustration of the zero-shot procedure for prototypical networks as it relates to the few-shot procedure is shown in Figure 1. Since the meta-data vector and query point come from different input domains, we found it was helpful empirically to fix the prototype embedding g to have unit length, however we do not constrain the query embedding f .

3 Experiments

For few-shot learning, we performed experiments on Omniglot [16] and the *miniImageNet* version of ILSVRC-2012 [26] with the splits proposed by Ravi and Larochelle [22]. We perform zero-shot experiments on the 2011 version of the Caltech UCSD bird dataset (CUB-200 2011) [31].

3.1 Omniglot Few-shot Classification

Omniglot [16] is a dataset of 1623 handwritten characters collected from 50 alphabets. There are 20 examples associated with each character, where each example is drawn by a different human subject. We follow the procedure of Vinyals et al. [29] by resizing the grayscale images to 28×28 and augmenting the character classes with rotations in multiples of 90 degrees. We use 1200 characters plus rotations for training (4,800 classes in total) and the remaining classes, including rotations, for test. Our embedding architecture mirrors that used by Vinyals et al. [29] and is composed of four convolutional blocks. Each block comprises a 64-filter 3×3 convolution, batch normalization layer [10], a ReLU nonlinearity and a 2×2 max-pooling layer. When applied to the 28×28 Omniglot images this architecture results in a 64-dimensional output space. We use the same encoder for embedding both support and query points. All of our models were trained via SGD with Adam [11]. We used an initial learning rate of 10^{-3} and cut the learning rate in half every 2000 episodes. No regularization was used other than batch normalization.

We trained prototypical networks using Euclidean distance in the 1-shot and 5-shot scenarios with training episodes containing 60 classes and 5 query points per class. We found that it is advantageous to match the training-shot with the test-shot, and to use more classes (higher “way”) per training episode rather than fewer. We compare against various baselines, including the neural statistician [6] and both the fine-tuned and non-fine-tuned versions of matching networks [29]. We computed classification accuracy for our models averaged over 1000 randomly generated episodes from the test set. The results are shown in Table 1 and to our knowledge they represent the state-of-the-art on this dataset.

3.2 *miniImageNet* Few-shot Classification

The *miniImageNet* dataset, originally proposed by Vinyals et al. [29], is derived from the larger ILSVRC-12 dataset [26]. The splits used by Vinyals et al. [29] consist of 60,000 color images of size 84×84 divided into 100 classes with 600 examples each. For our experiments, we use the splits introduced by Ravi and Larochelle [22] in order to directly compare with state-of-the-art algorithms for few-shot learning. Their splits use a different set of 100 classes, divided into 64 training, 16 validation, and 20 test classes. We follow their procedure by training on the 64 training classes and using the 16 validation classes for monitoring generalization performance only.

We use the same four-block embedding architecture as in our Omniglot experiments, though here it results in a 1600-dimensional output space due to the increased size of the images. We also

Table 2: Few-shot classification accuracies on *miniImageNet*. All accuracy results are averaged over 600 test episodes and are reported with 95% confidence intervals. *Results reported by [22].

Model	Dist.	Fine Tune	5-way Acc.	
			1-shot	5-shot
BASILINE NEAREST NEIGHBORS*	Cosine	N	28.86 $\pm$ 0.54%	49.79 $\pm$ 0.79%
MATCHING NETWORKS [29]*	Cosine	N	43.40 $\pm$ 0.78%	51.09 $\pm$ 0.71%
MATCHING NETWORKS FCE [29]*	Cosine	N	43.56 $\pm$ 0.84%	55.31 $\pm$ 0.73%
META-LEARNER LSTM [22]*	-	N	43.44 $\pm$ 0.77%	60.60 $\pm$ 0.71%
PROTOTYPICAL NETWORKS (OURS)	Euclid.	N	49.42 $\pm$ 0.78%	68.20 $\pm$ 0.66%

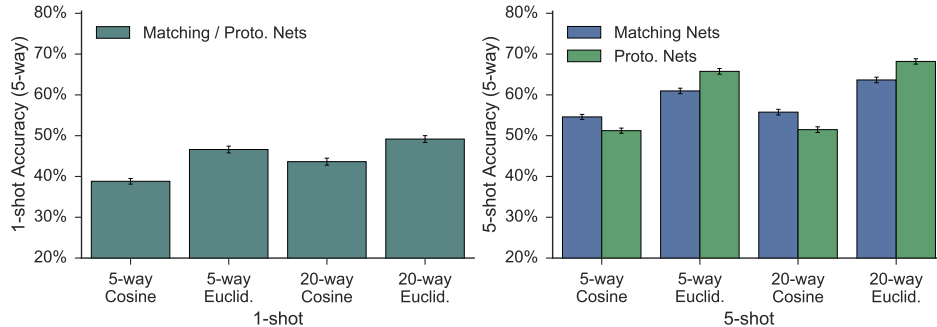


Figure 2: Comparison showing the effect of distance metric and number of classes per training episode on 5-way classification accuracy for both matching and prototypical networks on *miniImageNet*. The x -axis indicates configuration of the training episodes (way, distance, and shot), and the y -axis indicates 5-way test accuracy for the corresponding shot. Error bars indicate 95% confidence intervals as computed over 600 test episodes. Note that matching networks and prototypical networks are identical in the 1-shot case.

use the same learning rate schedule as in our Omniglot experiments and train until validation loss stops improving. We train using 30-way episodes for 1-shot classification and 20-way episodes for 5-shot classification. We match train shot to test shot and each class contains 15 query points per episode. We compare to the baselines as reported by Ravi and Larochelle [22], which include a simple nearest neighbor approach on top of features learned by a classification network on the 64 training classes. The other baselines are two non-fine-tuned variants of matching networks (both ordinary and FCE) and the Meta-Learner LSTM. As can be seen in Table 2, prototypical networks achieves state-of-the-art here by a wide margin.

We conducted further analysis, to determine the effect of distance metric and the number of training classes per episode on the performance of prototypical networks and matching networks. To make the methods comparable, we use our own implementation of matching networks that utilizes the same embedding architecture as our prototypical networks. In Figure 2 we compare cosine vs. Euclidean distance and 5-way vs. 20-way training episodes in the 1-shot and 5-shot scenarios, with 15 query points per class per episode. We note that 20-way achieves higher accuracy than 5-way and conjecture that the increased difficulty of 20-way classification helps the network to generalize better, because it forces the model to make more fine-grained decisions in the embedding space. Also, using Euclidean distance improves performance substantially over cosine distance. This effect is even more pronounced for prototypical networks, in which computing the class prototype as the mean of embedded support points is more naturally suited to Euclidean distances since cosine distance is not a Bregman divergence.

3.3 CUB Zero-shot Classification

In order to assess the suitability of our approach for zero-shot learning, we also run experiments on the Caltech-UCSD Birds (CUB) 200-2011 dataset [31]. The CUB dataset contains 11,788 images of 200 bird species. We closely follow the procedure of Reed et al. [23] in preparing the data. We use

Table 3: Zero-shot classification accuracies on CUB-200.

Model	Image Features	50-way Acc. 0-shot
ALE [1]	Fisher	26.9%
SJE [2]	AlexNet	40.3%
SAMPLE CLUSTERING [17]	AlexNet	44.3%
SJE [2]	GoogLeNet	50.1%
DS-SJE [23]	GoogLeNet	50.4%
DA-SJE [23]	GoogLeNet	50.9%
PROTO. NETS (OURS)	GoogLeNet	54.6%

their splits to divide the classes into 100 training, 50 validation, and 50 test. For images we use 1,024-dimensional features extracted by applying GoogLeNet [28] to middle, upper left, upper right, lower left, and lower right crops of the original and horizontally-flipped image². At test time we use only the middle crop of the original image. For class meta-data we use the 312-dimensional continuous attribute vectors provided with the CUB dataset. These attributes encode various characteristics of the bird species such as their color, shape, and feather patterns.

We learned a simple linear mapping on top of both the 1024-dimensional image features and the 312-dimensional attribute vectors to produce a 1,024-dimensional output space. For this dataset we found it helpful to normalize the class prototypes (embedded attribute vectors) to be of unit length, since the attribute vectors come from a different domain than the images. Training episodes were constructed with 50 classes and 10 query images per class. The embeddings were optimized via SGD with Adam at a fixed learning rate of 10^{-4} and weight decay of 10^{-5} . Early stopping on validation loss was used to determine the optimal number of epochs for retraining on the training plus validation set.

Table 3 shows that we achieve state-of-the-art results by a large margin when compared to methods utilizing attributes as class meta-data. We compare our method to other embedding approaches, such as ALE [1], SJE [2], and DS-SJE/DA-SJE [23]. We also compare to a recent clustering approach [17] which trains an SVM on a learned feature space obtained by fine-tuning AlexNet [14]. These zero-shot classification results demonstrate that our approach is general enough to be applied even when the data points (images) are from a different domain relative to the classes (attributes).

4 Related Work

The literature on metric learning is vast [15, 5]; we summarize here the work most relevant to our proposed method. Neighborhood Components Analysis (NCA) [8] learns a Mahalanobis distance to maximize K-nearest-neighbor’s (KNN) leave-one-out accuracy in the transformed space. Salakhutdinov and Hinton [27] extend NCA by using a neural network to perform the transformation. Large margin nearest neighbor (LMNN) classification [30] also attempts to optimize KNN accuracy but does so using a hinge loss that encourages the local neighborhood of a point to contain other points with the same label. The DNet-KNN [21] is another margin-based method that improves upon LMNN by utilizing a neural network to perform the embedding instead of a simple linear transformation. Of these, our method is most similar to the non-linear extension of NCA [27] because we use a neural network to perform the embedding and we optimize a softmax based on Euclidean distances in the transformed space, as opposed to a margin loss. A key distinction between our approach and non-linear NCA is that we form a softmax directly over *classes*, rather than individual points, computed from distances to each class’s prototype representation. This allows each class to have a concise representation independent of the number of data points and obviates the need to store the entire support set to make predictions.

Our approach is also similar to the nearest class mean approach [19], where each class is represented by the mean of its examples. This approach was developed to rapidly incorporate new classes into a classifier without retraining, however it relies on a linear embedding and was designed to handle

²Features downloaded from <https://github.com/reedscot/cvpr2016>.

the case where the novel classes come with a large number of examples. In contrast, our approach utilizes neural networks to non-linearly embed points and we couple this with episodic training in order to handle the few-shot scenario. Mensink et al. attempt to extend their approach to also perform non-linear classification, but they do so by allowing classes to have multiple prototypes. They find these prototypes in a pre-processing step by using k -means on the input space and then perform a multi-modal variant of their linear embedding. Prototypical networks, on the other hand, learn a non-linear embedding in an end-to-end manner with no such pre-processing, producing a non-linear classifier that still only requires one prototype per class. In addition, our approach naturally generalizes to other distance functions, particularly Bregman divergences.

Another relevant few-shot learning method is the meta-learning approach proposed in Ravi and Larochelle [22]. The key insight here is that LSTM dynamics and gradient descent can be written in effectively the same way. An LSTM can then be trained to itself train a model from a given episode, with the performance goal of generalizing well on the query points. Matching networks and prototypical networks can also be seen as forms of meta-learning, in the sense that they produce simple classifiers dynamically from new training episodes; however the core embeddings they rely on are fixed after training. The FCE extension to matching nets involves a secondary embedding that depends on the support set. However, in the few-shot scenario the amount of data is so small that a simple inductive bias seems to work well, without the need to learn a custom embedding for each episode.

Prototypical networks are also related to the neural statistician [6] from the generative modeling literature, which extends the variational autoencoder [12, 24] to learn generative models of datasets rather than individual points. One component of the neural statistician is the “statistic network” which summarizes a set of data points into a statistic vector. It does this by encoding each point within a dataset, taking a sample mean, and applying a post-processing network to obtain an approximate posterior over the statistic vector. Edwards and Storkey test their model for one-shot classification on the Omniglot dataset by considering each character to be a separate dataset and making predictions based on the class whose approximate posterior over the statistic vector has minimal KL-divergence from the posterior inferred by the test point. Like the neural statistician, we also produce a summary statistic for each class. However, ours is a discriminative model, as befits our discriminative task of few-shot classification.

With respect to zero-shot learning, the use of embedded meta-data in prototypical networks resembles the method of [3] in that both predict the weights of a linear classifier. The DS-SJE and DA-SJE approach of [23] also learns deep multimodal embedding functions for images and class meta-data. Unlike ours, they learn using an empirical risk loss. Neither [3] nor [23] uses episodic training, which allows us to help speed up training and regularize the model.

5 Conclusion

We have proposed a simple method called prototypical networks for few-shot learning based on the idea that we can represent each class by the mean of its examples in a representation space learned by a neural network. We train these networks to specifically perform well in the few-shot setting by using episodic training. The approach is far simpler and more efficient than recent meta-learning approaches, and produces state-of-the-art results even without sophisticated extensions developed for matching networks (although these can be applied to prototypical nets as well). We show how performance can be greatly improved by carefully considering the chosen distance metric, and by modifying the episodic learning procedure. We further demonstrate how to generalize prototypical networks to the zero-shot setting, and achieve state-of-the-art results on the CUB-200 dataset. A natural direction for future work is to utilize Bregman divergences other than squared Euclidean distance, corresponding to class-conditional distributions beyond spherical Gaussians. We conducted preliminary explorations of this, including learning a variance per dimension for each class. This did not lead to any empirical gains, suggesting that the embedding network has enough flexibility on its own without requiring additional fitted parameters per class. Overall, the simplicity and effectiveness of prototypical networks makes it a promising approach for few-shot learning.

Acknowledgements

We would like to thank Marc Law, Sachin Ravi, Hugo Larochelle, Renjie Liao, and Oriol Vinyals for helpful discussions. This work was supported by the Samsung GRP project and the Canadian Institute for Advanced Research.

References

- [1] Zeynep Akata, Florent Perronnin, Zaid Harchaoui, and Cordelia Schmid. Label-embedding for attribute-based classification. In *Computer Vision and Pattern Recognition*, pages 819–826, 2013.
- [2] Zeynep Akata, Scott Reed, Daniel Walter, Honglak Lee, and Bernt Schiele. Evaluation of output embeddings for fine-grained image classification. In *Computer Vision and Pattern Recognition*, pages 2927–2936, 2015.
- [3] Jimmy Ba, Kevin Swersky, Sanja Fidler, and Ruslan Salakhutdinov. Predicting deep zero-shot convolutional neural networks using textual descriptions. In *International Conference on Computer Vision*, pages 4247–4255, 2015.
- [4] Arindam Banerjee, Srjana Merugu, Inderjit S Dhillon, and Joydeep Ghosh. Clustering with bregman divergences. *Journal of machine learning research*, 6(Oct):1705–1749, 2005.
- [5] Aurélien Bellet, Amaury Habrard, and Marc Sebban. A survey on metric learning for feature vectors and structured data. *arXiv preprint arXiv:1306.6709*, 2013.
- [6] Harrison Edwards and Amos Storkey. Towards a neural statistician. *International Conference on Learning Representations*, 2017.
- [7] Mohamed Elhoseiny, Babak Saleh, and Ahmed Elgammal. Write a classifier: Zero-shot learning using purely textual descriptions. In *International Conference on Computer Vision*, pages 2584–2591, 2013.
- [8] Jacob Goldberger, Geoffrey E. Hinton, Sam T. Roweis, and Ruslan Salakhutdinov. Neighbourhood components analysis. In *Advances in Neural Information Processing Systems*, pages 513–520, 2004.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [10] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [11] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [12] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [13] Gregory Koch. Siamese neural networks for one-shot image recognition. Master’s thesis, University of Toronto, 2015.
- [14] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, pages 1097–1105, 2012.
- [15] Brian Kulis. Metric learning: A survey. *Foundations and Trends in Machine Learning*, 5(4):287–364, 2012.
- [16] Brenden M. Lake, Ruslan Salakhutdinov, Jason Gross, and Joshua B. Tenenbaum. One shot learning of simple visual concepts. In *CogSci*, 2011.
- [17] Renjie Liao, Alexander Schwing, Richard Zemel, and Raquel Urtasun. Learning deep parsimonious representations. *Advances in Neural Information Processing Systems*, 2016.
- [18] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.
- [19] Thomas Mensink, Jakob Verbeek, Florent Perronnin, and Gabriela Csurka. Distance-based image classification: Generalizing to new classes at near-zero cost. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(11):2624–2637, 2013.

- [20] Erik G Miller, Nicholas E Matsakis, and Paul A Viola. Learning from one example through shared densities on transforms. In *CVPR*, volume 1, pages 464–471, 2000.
- [21] Renqiang Min, David A Stanley, Zineng Yuan, Anthony Bonner, and Zhaolei Zhang. A deep non-linear feature mapping for large-margin knn classification. In *IEEE International Conference on Data Mining*, pages 357–366, 2009.
- [22] Sachin Ravi and Hugo Larochelle. Optimization as a model for few-shot learning. *International Conference on Learning Representations*, 2017.
- [23] Scott Reed, Zeynep Akata, Bernt Schiele, and Honglak Lee. Learning deep representations of fine-grained visual descriptions. *arXiv preprint arXiv:1605.05395*, 2016.
- [24] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. *arXiv preprint arXiv:1401.4082*, 2014.
- [25] Oren Rippel, Manohar Paluri, Piotr Dollar, and Lubomir Bourdev. Metric learning with adaptive density discrimination. *International Conference on Learning Representations*, 2016.
- [26] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.
- [27] Ruslan Salakhutdinov and Geoffrey E. Hinton. Learning a nonlinear embedding by preserving class neighbourhood structure. In *AISTATS*, pages 412–419, 2007.
- [28] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–9, 2015.
- [29] Oriol Vinyals, Charles Blundell, Tim Lillicrap, Daan Wierstra, et al. Matching networks for one shot learning. In *Advances in Neural Information Processing Systems*, pages 3630–3638, 2016.
- [30] Kilian Q Weinberger, John Blitzer, and Lawrence K Saul. Distance metric learning for large margin nearest neighbor classification. In *Advances in Neural Information Processing Systems*, pages 1473–1480, 2005.
- [31] P. Welinder, S. Branson, T. Mita, C. Wah, F. Schroff, S. Belongie, and P. Perona. Caltech-UCSD Birds 200. Technical Report CNS-TR-2010-001, California Institute of Technology, 2010.

A Additional Omniglot Results

In Table 4 we show test classification accuracy for prototypical networks using Euclidean distance trained with 5, 20, and 60 classes per episode.

Table 4: Additional classification accuracy results for prototypical networks on Omniglot. Configuration of training episodes is indicated by number of classes per episode (“way”), number of support points per class (“shot”) and number of query points per class (“query”). Classification accuracy was averaged over 1,000 randomly generated episodes from the test set.

Model	Dist.	Train Episodes			5-way Acc.		20-way Acc.	
		Shot	Query	Way	1-shot	5-shot	1-shot	5-shot
PROTONETS	Euclid.	1	15	5	97.4%	99.3%	92.0%	97.8%
PROTONETS	Euclid.	1	15	20	98.7%	99.6%	95.4%	98.8%
PROTONETS	Euclid.	1	5	60	98.8%	99.7%	96.0%	99.0%
PROTONETS	Euclid.	5	15	5	96.9%	99.3%	90.7%	97.8%
PROTONETS	Euclid.	5	15	20	98.1%	99.6%	94.1%	98.7%
PROTONETS	Euclid.	5	5	60	98.5%	99.7%	94.7%	98.9%

Figure 3 shows a sample t-SNE visualization [18] of the embeddings learned by prototypical networks. We visualize a subset of test characters from the same alphabet in order to gain better insight, despite the fact that classes in actual test episodes are likely to come from different alphabets. Even though the visualized characters are minor variations of each other, the network is able to cluster the hand-drawn characters closely around the class prototypes.

B Additional *mini*ImageNet Results

In Table 5 we show the full results for the comparison of training episode configuration in Figure 2 of the main paper.

We also compared Euclidean-distance prototypical networks trained with a different number of classes per episode. Here we vary the classes per training episode from 5 up to 30 while keeping the number of query points per class fixed at 15. The results are shown in Figure 4. Our findings indicate that construction of training episodes is an important consideration in order to achieve good results for few-shot classification. Table 6 contains the full results for this set of experiments.

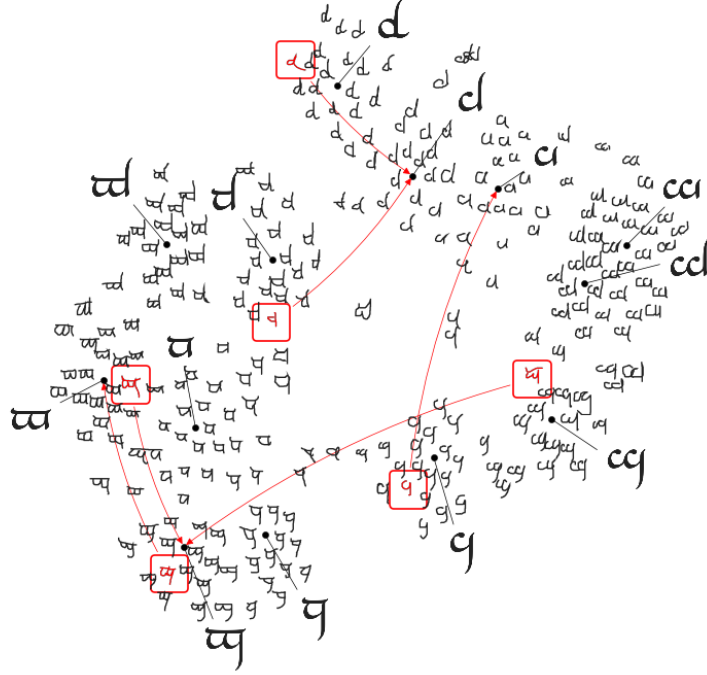


Figure 3: A t-SNE visualization of the embeddings learned by prototypical networks on the Omniglot dataset. A subset of the Tengwar script is shown (an alphabet in the test set). Class prototypes are indicated in black. Several misclassified characters are highlighted in red along with arrows pointing to the correct prototype.

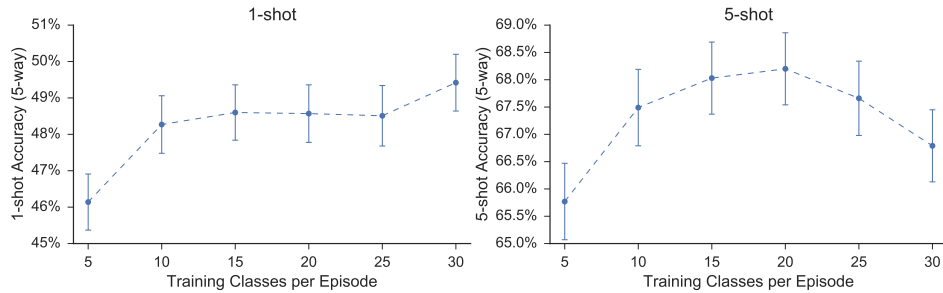


Figure 4: Comparison of the effect of training “way” (number of classes per episode) for prototypical networks trained on *miniImageNet*. Each training episode contains 15 query points per class. Error bars indicate 95% confidence intervals as computed over 600 test episodes.

Table 5: Comparison of matching and prototypical networks on *miniImageNet* under cosine vs. Euclidean distance, 5-way vs. 20-way, and 1-shot vs. 5-shot. All experiments use a shared encoder for both support and query points with embedding dimension 1,600 (architecture and training details are provided in Section 3.2 of the main paper). Classification accuracy is averaged over 600 randomly generated episodes from the test set and 95% confidence intervals are shown.

Model	Dist.	Train Episodes			5-way Acc.	
		Shot	Query	Way	1-shot	5-shot
MATCHING NETS / PROTONETS	Cosine	1	15	5	38.82 $\pm$ 0.69%	44.54 $\pm$ 0.56%
MATCHING NETS / PROTONETS	Euclid.	1	15	5	46.61 $\pm$ 0.78%	59.84 $\pm$ 0.64%
MATCHING NETS / PROTONETS	Cosine	1	15	20	43.63 $\pm$ 0.76%	51.34 $\pm$ 0.64%
MATCHING NETS / PROTONETS	Euclid.	1	15	20	49.17 $\pm$ 0.83%	62.66 $\pm$ 0.71%
MATCHING NETS	Cosine	5	15	5	46.43 $\pm$ 0.74%	54.60 $\pm$ 0.62%
MATCHING NETS	Euclid.	5	15	5	46.43 $\pm$ 0.78%	60.97 $\pm$ 0.67%
MATCHING NETS	Cosine	5	15	20	46.46 $\pm$ 0.79%	55.77 $\pm$ 0.69%
MATCHING NETS	Euclid.	5	15	20	47.99 $\pm$ 0.79%	63.66 $\pm$ 0.68%
PROTONETS	Cosine	5	15	5	42.48 $\pm$ 0.74%	51.23 $\pm$ 0.63%
PROTONETS	Euclid.	5	15	5	44.53 $\pm$ 0.76%	65.77 $\pm$ 0.70%
PROTONETS	Cosine	5	15	20	42.45 $\pm$ 0.73%	51.48 $\pm$ 0.70%
PROTONETS	Euclid.	5	15	20	43.57 $\pm$ 0.82%	68.20 $\pm$ 0.66%

Table 6: Effect of training “way” (number of classes per training episode) for prototypical networks with Euclidean distance on *miniImageNet*. The number of query points per class in training episodes was fixed at 15. Classification accuracy is averaged over 600 randomly generated episodes from the test set and 95% confidence intervals are shown.

Model	Dist.	Train Episodes			5-way Acc.	
		Shot	Query	Way	1-shot	5-shot
PROTONETS	Euclid.	1	15	5	46.14 $\pm$ 0.77%	61.36 $\pm$ 0.68%
PROTONETS	Euclid.	1	15	10	48.27 $\pm$ 0.79%	64.18 $\pm$ 0.68%
PROTONETS	Euclid.	1	15	15	48.60 $\pm$ 0.76%	64.62 $\pm$ 0.66%
PROTONETS	Euclid.	1	15	20	48.57 $\pm$ 0.79%	65.04 $\pm$ 0.69%
PROTONETS	Euclid.	1	15	25	48.51 $\pm$ 0.83%	64.63 $\pm$ 0.69%
PROTONETS	Euclid.	1	15	30	49.42 $\pm$ 0.78%	65.38 $\pm$ 0.68%
PROTONETS	Euclid.	5	15	5	44.53 $\pm$ 0.76%	65.77 $\pm$ 0.70%
PROTONETS	Euclid.	5	15	10	45.09 $\pm$ 0.79%	67.49 $\pm$ 0.70%
PROTONETS	Euclid.	5	15	15	44.07 $\pm$ 0.80%	68.03 $\pm$ 0.66%
PROTONETS	Euclid.	5	15	20	43.57 $\pm$ 0.82%	68.20 $\pm$ 0.66%
PROTONETS	Euclid.	5	15	25	43.32 $\pm$ 0.79%	67.66 $\pm$ 0.68%
PROTONETS	Euclid.	5	15	30	41.38 $\pm$ 0.81%	66.79 $\pm$ 0.66%